

When Is a Numerological Search Finished? An Exhaustive Null to Depth 10, and What Went Wrong With Our Own Stopping Rule

C. P. Zimmerman Briar Creek Tech · Charlotte, NC

v3, 2026-10-08. Supersedes v2 (2026-07-29, Zenodo DOI 10.5281/zenodo.21654272). v3 is a correction release: an independent claim-by-claim audit traced every number in v2 to the outputs the scripts write (§11). The headline null and every conclusion stand. Twelve statements carried wrong numbers, six overstated what the outputs show, and seven numbers had no saved output. Those scripts were re-run and their outputs are now saved. All are corrected below. v2 itself superseded the unpublished v1 draft, of which five load-bearing claims were corrected and two were false.

Abstract

Searches for closed-form expressions reproducing Standard-Model dimensionless constants are common and almost never conclude: a null is reported as “we did not find one,” which is not a statement about the space. We set out to do two things about that for one concrete search, and the second one failed instructively.

First, a worked exhaustive null — this stands. For a modified-inertia gravity framework whose acceleration scale is tied to the cosmological constant, $a_0 = cH_{\text{Lambda}}/Z$ with $Z = \sqrt{32\pi/3}$, we exhaustively enumerate every dimensionless value constructible at depth ≤ 10 from the framework’s two germs — the generation count 3 and the kernel germ $\sqrt{8\pi/3}$ — and confront all of them with 19 measured SM targets through three gates. Result: **174,890,804 raw candidates, 42,534,139 distinct values, 82,613 in-window hits, ZERO surviving the gate.** Depths 6–9 are exhaustive clean nulls under the same engine. Depths 3 and 5 are clean nulls under the earlier forced-exhaustion engines (21 targets). Depth 4 is empty there by a depth-budget theorem (§2). Depth 10 extends the proven-empty range and is the first depth past 9 where the word *exhaustive* applies rather than *sampled*. This result is untouched by every correction below, because the operative gate never used the quantity we got wrong (§7).

Second, a depth ceiling — the qualitative claim stands, our formula did not. The diagnostic is *where the hits land*: hit count tracks measurement-window width and nothing else. The six most precisely measured targets return **exactly zero** hits across 42.5 million values; the loosely measured neutrino mixing angles return 15,000–26,000. Single-target matching therefore stops being informative at a computable depth, and our sampling campaign at depths 10–18 was statistically empty. But the ceiling formula we published to express this, $D_{\text{max}} = D_0 + \ln(1/w)/\ln(B)$ with $B = 30$, $D_0 = 4$, is wrong in both parameters: **$B = 30$ is the step-menu length, not a branching factor — the realized factor is 4.41**, and $D_0 = 4$ is unfindable because depth 4 cannot be built in this engine. The corrected ceiling for $1/\alpha$ is **$D = 11.0$** (we published 10.4); the level survived only by cancellation of four errors, while the slope was wrong by 2.5x.

Third, and this is the paper’s main methodological content: our proposed way past the ceiling was pricing an object that cannot exist. We recommended that discriminating power come from *simultaneous* matching across sectors, with a threshold in bits, $\text{SUM}_i \log_2(1/w_i) > \log_2(N(D)) + \text{margin}$. That charges the look-elsewhere cost **once** while crediting the information of k targets — which describes one expression matching k constants at once. A single expression evaluates to a single number and distinct targets have disjoint windows, so no expression can ever do that. The object a search of this form actually retains is a **skeleton** (a structural template) reaching k targets via k *different* germ recipes, so the cost is incurred k times, not once:

$$\text{cost}(k, D) = \log_2(N_{\text{skel}}(D)) + k \cdot \log_2(R_{\text{recipe}}(D)), \quad \text{avail}(k) = \text{SUM}_i \log_2(1/2w_i)$$

Adding a target buys its bits **and** another full recipe search. With the missing term restored, $k_{\min} = 4$ at depth 18 rather than the 2 or 3 the two readings of the old rule gave. A label-permutation null on the real records is harsher still: chance alone puts a skeleton on **~11 of the 18 independent targets simultaneously**, real multiplicities sit *below* chance at every k ($z = -15.3$ to -0.1), and no k up to 12 is rare enough to clear a family-wise threshold. An apparent 11-target interlock in this search would be noise.

What is not claimed. The look-elsewhere argument is standard (Gross & Vitells 2010); nothing here is new statistics. The null is about one germ vocabulary at one depth under one gate. It is **not** evidence for or against the framework whose germs it uses: that framework’s acceleration scale, the constant Z , the response sign, and its gate frequency remain postulated, not derived, and this search does not bear on them. Two claims from the v1 draft are **withdrawn as false**: that the search had a valid held-back validation set (§6 — both holdout targets are exact algebraic functions of targets the search may fit), and that requiring the germs made the search falsifiable (§1 — on the enumerated path that requirement cannot fail).

1. What was searched, and the withdrawal of “forced”

The framework supplies exactly two germs and one free $O(1)$ coefficient:

germ	origin in the framework
3	the generation count
$\sqrt{8\pi/3}$	the kernel germ, half of $Z = \sqrt{32\pi/3}$

Expressions are built by constructive enumeration: a base leaf followed by up to $D-4$ skeleton steps (at $D = 10$ the skeleton takes $b_s = 1-6$ steps, the rest of the budget going to germ recipes) from a fixed 30-entry menu (leaf append with MUL/DIV, five power exponents, three unaries), decorated by canonical germ recipes. Value-identical expressions are deduplicated at mpmath precision, so the object counted is *distinct values*, not distinct formulas.

WITHDRAWN. The v1 draft said: “*Forced’ is load-bearing and is what makes the search falsifiable rather than open-ended. A candidate expression must contain the germs (Gate B)...*”

Without that requirement the space is unbounded and no null means anything.” Two corrections (audit_interlock/gate_b_germ_enforcement.py, gate_b_cancelling_germ.py):

1. **On the enumerated path Gate B is a constructive tautology, not a filter.** The germ layer emits both forced germs plus exactly one free germ *by construction*, so no enumerated candidate can fail the germ-content test. Against germ *absence* in the candidates the enumerator can produce, the gate behaves as intended: 9/9 unit probes pass (no-germ FAIL, only-3 FAIL, only-sqrt(8pi/3) FAIL, both+1free PASS; gate_b_germ_enforcement_output.txt, Q1). It never binds on the path the search takes. It is not airtight, though. Hand-fed candidates that *self-declare* forced appearances can clear it without the sqrt(8pi/3) germ, or with zero forced factors (probes PA-PD, Q2). These holes are unreachable from the enumerator (Q3), so the null is unaffected, but “the code is right” would overstate it.
2. **A germ is credited by syntactic presence, not by being load-bearing.** A germ whose net signed exponent is zero — multiplied then divided, i.e. algebraically cancelled — still counts as present.

So the honest statement is weaker: the enumeration is *restricted* to a germ-decorated vocabulary by construction. That bounds the space, which is what makes the null finite and exhaustive. It does not additionally certify that any surviving expression would have used the germs essentially.

Targets. 19 measured dimensionless SM quantities, with relative half-widths from 1.1e-10 to 4.0e-2: about 8.5 orders of magnitude. Two further targets were nominally held back; §6 withdraws that.

2. The exhaustive null at depth ≤ 10 — unchanged

sharded_build.py, sweep_depth.py, NULL_RESULT_DEPTH10_EXHAUSTIVE.md.

```
raw candidates enumerated : 174,890,804
distinct values           : 42,534,139   (33,309,838 distinguishable in float64)
targets swept             : 19
in-window hits            : 82,613
CERTIFIED (gate-passing) : 0
RE-LABELED                : 0
```

Depths 6–9 are exhaustive clean nulls under the same engine (grind.py; committed per-depth verdicts). The depth sets are **nested**: depths 8 and 9 lie 100% inside depth 10, and $|\text{union}(D8, D9, D10)| = |D10| = 33,309,840$ (effective_N_audit_output.txt; ceiling_math_audit_output.txt counts 33,309,838 float64-distinct, a 2-value disagreement between the two audit scripts that moves nothing). So $\text{distinct}(D)$ is a fair total with no union inflation. Depths 3 and 5 were exhausted earlier by the forced-exhaustion engines over 21 targets with a 25-germ FDR. Those are different machinery: depth-5 raw is 918,528 there against 19,136 under grind.py. In the forced engine, depth 4 admits no dimensionless expression at all, by a depth-budget theorem: a Gate-B-passable candidate needs three germ leaves, which leaves room for exactly one dimensionful scale leaf. Re-run on 2026-10-08 over the current 19 targets: 777,216 Gate-B-passable trees per target, 0 dimensionally valid values, 0 hits (results_exhaust_depth4/sweep_output.txt). VERDICT_depth4_forced.md records a July run over 21 targets, but that run’s output was not

saved. In `grind.py`, depth 4 cannot be built (`budget_splits(4)` is empty). An exact identity reproduces every committed raw count, `raw(D) = SUM_splits n_skel(b_s) * n_recipes(g_s)` with `n_skel = {13, 73, 247, 1147, 5250, 22708}`.

3. The hit distribution is the result, not the null — confirmed at higher precision

target	rel. half-width w	hits
$m_p/m_e, a_e, 1/\alpha, m_n/m_p$	$1.1e-10 - 6.5e-10$	0
$m_\mu/m_e, a_\mu$	$2.2e-8 - 1.9e-7$	0
$r_{\tau_e}, 1/\alpha(M_Z), \sin^2 \theta_W$	$6.8e-5 - 1.3e-4$	28 / 50 / 72
$koide_Q_{up}, higgs_lambda, ckm_lambda, koide_Q_{down}$	$1.3e-3 - 3.1e-3$	838 / 1130 / 2121 / 2098
$r_{b_tau}, r_{t_b}, \alpha_s(M_Z)$	$7.2e-3 - 7.6e-3$	4747 / 4443 / 4933
$pmns_sin2_{13/12/23}$	$2.5e-2 - 4.0e-2$	15212 / 26142 / 20799

(Half-widths w and hits from `audit_interlock/expected_count_audit.json.v2` printed a mix of half- and full-width figures in this column; the hit counts were right.)

Every one of the 82,613 died in the false-discovery gate. Re-measured directly off the committed value arrays, `hits/2w` is **flat at $\rho = 2.1e5 - 3.6e5$ across the 13 targets that have hits**, whose windows span ~ 2.8 decades. For the six tight targets, $\rho \cdot 2w$ predicts well under one hit, and they show zero. Hit count tracks window width and nothing else.

A correction to v1's diagnosis, with the number intact. The naive expected-count model $N \cdot 2w$ overpredicts observed hits by a factor of **125** (naive 10,328,960 vs observed 82,613; median 124.6x over the 13 swept targets with hits, or 123x if the held-back `koide_Q_lep` is included. Every recorded per-target count reproduces exactly, 13/13. That holds only after fixing a key mismatch in `expected_count_audit.py`: in the copy committed with v2 (zimmerman-formula `cfb6007f`), the `sin^2 theta_W` row has `recorded = null`, so v2's check skipped it.) v1 attributed this to *clustering*. That is wrong: the factor is **dynamic range** — the value set spans 632 decades in $|v|$ (1454 natural-log units), so almost all values are nowhere near any target. At window scale the distribution is locally smooth. Two consequences, both measured (`audit_interlock/clustering_local_density.py`): the effect makes the threshold **conservative, not anti-conservative**, so no penalty is needed (net worst-clump bits +6.71). The measured targets are not on density spikes: the median enrichment over the 20 targets is 1.00x (47th percentile), and the largest is `r_mu_e` at 1.73x (98th).

4. The depth ceiling: the claim survives, the formula does not

`GATE_POWER_ANALYSIS.py`, `audit_interlock/ceiling_math_audit.py`, `effective_N_audit.py`.

The qualitative result is unchanged and was re-tested adversarially against the hypothesis that it reverses: **single-target matching dies by ~D11–13 and depth 18 is statistically empty**. With the corrected multiplicity, the expected chance count at depth 18 is 3.87 for $1/\alpha$, 10.6 for m_p/m_e and 3.02 for a_e — all far above any sane threshold. Our own depths 10–18 sampling campaign (1,191 distinct passes over depths 10–18, 18,332 in-window hits, every one FDR-DEAD; results_grind/SAMPLE_LEDGER.jsonl, where one resumed pass is logged twice) was incapable of establishing a single-target result *by construction*, which we did not appreciate until after running it.

But four things about the published formula were wrong.

1. **B = 30 is not a branching factor.** It is the literal length of the skeleton step menu — the enumerator’s *work*, not its yield. After the dimensionless/finite/positive filter the $b_s=6$ layer collapses from $8.019e9$ sequences to 22,708 skeletons, and the germ layer multiplies back only polynomially (recipe ratios fall $6.75 \rightarrow 1.61$). The realized per-depth factor is **4.41** on distinct values (3.82 on local density near a target), i.e. **1.94–2.14 bits/depth, not $\log_2(30) = 4.91$** . $30^{(D-4)}$ over-counts by 17x at D10, 794x at D12, and $7.9e7x$ at D18.
2. **D0 = 4 is unfoundable.** The model asserts $N(D0) = 1$, but `budget_splits(4)` is *empty*: $N(4) = 0$ and depth 4 cannot be built. The first non-empty depth is 5, with 19,136 raw.
3. **The look-elsewhere count must be distinct values, not raw candidates.** Value-identical expressions hit or miss together and contribute zero extra chances. Verified by re-enumerating depth 6 with multiplicity: raw 236,624, distinct 107,719, in-window hits 259 — matching the committed 259, so the deduplicated count *is* the statistic the pipeline reports. Dedup is worth 2.04 bits at D10, ~3.86 at D18.
4. **E = 1 is not an informativeness criterion** — it is a 63% false-alarm rate. A family-wise 5% threshold over 19 targets x 8 exhaustive depths gives $E^* = 3.3e-4$, which *lowers* $D_{\max}$ by ~6 depths. This is the one error in the conservative direction and it very nearly cancels the multiplicity errors.

Corrected ceilings, with the measured density, measured branching and family-wise E^* : $1/\alpha$ **D = 11.0** (published 10.4) and m_p/m_e **D = 10.3** on the window the code applies, 12.7 on the CODATA direct ratio (published 13.1). The error ladder for $1/\alpha$, substituting one correction at a time: $10.44 \rightarrow 11.28$ (real multiplicity level) $\rightarrow 12.93$ (real growth) $\rightarrow 16.99$ (measured local density) $\rightarrow$ **11.03** (family-wise E^*). The published *level* survived by cancellation; the *slope* was wrong by 2.5x, so every claim that depended on the slope is void — including “each additional depth costs 4.9 bits.”

Two window data errors. `GATE_POWER_ANALYSIS.py:40` writes $3.2e-11/1836.15$ for m_p/m_e . That is the absolute uncertainty entered 1000x too small: CODATA’s is $3.2e-8$. The window used was therefore **1000x too tight**. Correcting only that turns v1’s “44.7 bits for m_p/m_e ” into **34.7** on the CODATA ratio ($\log_2(1/(2 \times 1.743e-11))$). On the full window the search itself propagates ($2w = 8.5e-10$, about 24x looser again), it is **30.1**. m_{μ}/m_e carries a separate factor-10 error ($4.6e-7$ for $4.6e-6$, `GATE_POWER_ANALYSIS.py:41`; this is a source line, not an output). Consequently v1’s headline “the two most precisely measured numbers in physics supply 76.3 bits” is really **61.7** on

the search’s own windows. Both errors are in *advertised thresholds only* — the running gate reads windows from the dataset, so no committed verdict is affected.

5. The interlock rule priced an impossible object

THRESHOLD.py (new; supersedes BITS_RULE.py’s rule and GATE_POWER_ANALYSIS.py’s $k_{\min}$ table).

v1 recommended that all discriminating power at depth come from simultaneous matching, with the threshold $\text{SUM}_i \log_2(1/w_i) > \log_2(N(D)) + \text{margin}$. **This is the paper’s main error and its main lesson.** The rule charges the look-elsewhere cost once and credits k targets’ information, which describes *one expression matching k constants simultaneously*. An expression evaluates to one number; distinct SM targets have disjoint windows; this event has probability zero.

Verified on the committed records: the object the search actually retains is a **skeleton** reaching k targets via k **distinct germ recipes** — never one value. The cost is therefore paid k times:

```
cost(k, D) = log2(N_skel(D)) + k * log2(R_recipe(D))
avail(k)   = SUM over k independent targets of log2(1/2w_i)
informative <=> avail(k) > cost(k, D) + margin
```

The k -linear term is what both prior readings omitted, and it explains why the same data gave two different answers — $k_{\min} = 2$ under the corrected multiplicity, $k_{\min} = 3$ under the published one. That disagreement is **void**. With the term restored:

depth	$k_{\min}$
12	2
15	3
18	4

The empirical calibration is harsher, and it is the one that binds. A label-permutation null over the 78,170 assignable depth-10 records (2,651 skeletons), preserving both per-skeleton hit multiplicity and the lopsided per-target marginal:

k targets on one skeleton	real #skeletons	chance mean	z
2	2,269	2,360.5	−15.3
5	1,522	1,683.2	−13.1
8	442	511.9	−5.1
10	33	33.4	−0.1
11	0	2.6	−1.8
12	0	0.07	−0.3

Chance alone puts a skeleton on **~11 of the 18 independent targets at once** (mean chance maximum 11.03; results_interlock/interlock_depth10.log). The real maximum is 10. “Inde-

pendent” is the script’s label: its 18-target set still holds both a_e and $1/\alpha$, which are effectively one observable (below). Neither target has any hits, so the null does not move. Real multiplicities are *below* the null at every k . No k up to 12 is rare enough to clear $E^* = 3.3e-4$. So in a search of this shape an apparent 11-target interlock is not a discovery — and any future interlock claim must be calibrated against this null, not against analytic bits.

The interlock search, run properly, is also a null — and this one is validated. INTERLOCK_SEARCH.py implements the above over the maximal independent set (18 targets), calibrated by permutation null, and it **refuses to report a null until it passes controls**. A planted skeleton reaching $k = 18$ targets is recovered ($p = 0.0000$); a planted skeleton at $k = 10$, inside the chance band, is correctly *not* flagged ($p = 0.815$). Only then does it run. At depth 10: 78,170 of 82,624 records lie in an independent-set window, carried by 2,651 skeletons; a claim would need $k \geq 13$; the observed maximum is **10** against a chance maximum of 11.0, $p = 1.0000$. **Nothing survives.**

This ordering matters more than the result. v1 proposed the interlock remedy without ever demonstrating that a search built on it could recover a known interlock. Had we run the analytic rule instead, the $k = 11$ coincidences already present in the data would have been reported as an 85-bit discovery.

Interlock bits also need an independent target set. a_e and $1/\alpha$ are effectively one observable: the mass-independent QED series from α alone reproduces measured a_e to $2.4e-9$. They sit in *different* sector fields, so a sector count cannot catch it. Worked false positive: $k=3$ on $\{a_e, 1/\alpha, m_p/m_e\}$ advertises 93.8 bits, clears v1’s depth-18 threshold of 78.7, and has true independent content 65.2 — below (target_independence_graph_output.txt, which also gives the $2.4e-9$). Across the registry’s algebraic families, 35 member targets carry 17 independent degrees of freedom, so 18 are exactly redundant. Within the 19-target fittable pool, the worst correlation is $\rho(r_{b_tau}, r_{t_b}) = -0.974$ through the shared m_b . The interlock null is run on 18 targets with r_{t_b} dropped. This is conservative: the pair is functionally independent (Jacobian rank 2; interlock_spec_sets_output.txt T4), so dropping it costs power, never validity. One distinction must not be blurred: for *theory*-linked pairs the look-elsewhere bits **do** legitimately add, because the enumeration cannot build the QED series or RG running. a_e/α is a k -count error, not a bits-arithmetic error. Only *algebraic* edges make a hit free.

6. WITHDRAWN: the search had no valid held-back validation set

audit_interlock/target_independence_graph.py.

v1 §6 claimed two targets were excluded from every search pool so that a survivor “would have to *predict* them.” **This is false, and the reason is algebraic rather than a coding leak.**

```
r_tau_mu    = r_tau_e / r_mu_e                                (exactly)
koide_Q_lep = (1+A+B)/(1+sqrt(A)+sqrt(B))^2,  A = r_mu_e, B = r_tau_e
```

Both are exact functions of targets the search **is** allowed to fit. Measured: full-pool span residuals $1.686e-10$ and $3.300e-10$ (spanned); fitted exponents $(-1.0000, +1.0000)$; **transfer gain 1.000329 sigma per input sigma** (1.0 to three decimals); conditional informa-

tion **0.0000 bits** (target_independence_graph_output.txt). The repo’s own score_holdout passes $r_{\tau\mu}$ at 0.03 sigma for 16.8170, which is the pool quotient $r_{\tau e}/r_{\mu e}$ (holdout_airtight_output.txt). There is no slack anywhere for the holdout to test.

The name-based guard is intact and irrelevant — this survives a perfect name guard. Worse, the target v1 called “the stronger test” is unscorable: $r_{\tau\mu}$ has **zero retained records** at depth 10, because `sm_target_keys(include_holdout=True)` re-adds only `koide_Q_lep`.

v1 already reported against interest that `koide_Q_lep` sits 0.91 sigma from exact 2/3 and is therefore weak. The correct statement is stronger and worse: **this search has no valid holdout at all**. A genuine one must be algebraically independent of the fitted pool, and no such target exists among the lepton ratios. This is a general trap for constructive searches over an algebraically closed target set: any holdout reachable from the pool by the search’s own operations (MUL, DIV, SQRT are all in the menu) is free.

7. Why the null is unaffected by every correction above

`audit_interlock/effective_N_audit.py`.

`gate/fdr.py` as coded **never uses $N(D)$** . Its look-elsewhere multiplier is `mult = n_targets_searched` and its `E_chance` comes from `build_value_set` over a 25-germ mini-library, not from the enumeration. So the depth-10 clean null, every committed FDR-DEAD label, and the zero-certified result are untouched. The enumeration multiplicity was never charged operationally — only the v1 ceiling table, `GATE_POWER_ANALYSIS.py`’s `S1/S2/S6` and `BITS_RULE.py`’s read-out rule were wrong. Re-verified directly from values: `f64`: 0 in-window hits at $1/\alpha$ and `m_p/m_e` at depths 8, 9 and 10.

Also resolved: the 42,534,139 published distinct count versus 40,304,591 in the audit is not a discrepancy. Float64 rails remove exactly 2,229,548 values ($42,534,139 - 2,229,548 = 40,304,591$), and a further 6,994,753 collide, leaving 33,309,838 float64-distinguishable.

8. Why nothing was there — four independent structural obstructions

Established separately; each would stand without the search.

1. **Number field.** Z carries a transcendental $\sqrt{\pi}$; the flavour and coupling data are algebraic. An exact identity requires the $\sqrt{\pi}$ to cancel — in which case the germ was not load-bearing.
2. **Period ring.** Half-integer versus integer weight, with the weight-1 slot empty: the rings are disjoint exactly where a bridge would live.
3. **Dictionary/category.** $u = a_0/g$ is an acceleration *ratio*; a renormalisation scale is an *energy*. The only dimensional bridge, $a_0/2c$ with $\hbar$ and c , lands 39.7 orders below the electron (`rg_flow_correspondence_log.json`; that log’s verdict string, which says “~24”, is wrong).
4. **Varying constants — the one experimental closure.** If any SM constant tracked the dark-energy density as strongly as a_0 does, atomic clocks would have seen it; the coupling is bounded to $|p| \leq 6e-8$.

9. Method: three bugs the guards caught

Recorded because the guards are why the null is trustworthy.

1. **A merge that re-keyed float64** silently discarded 19% of depth 8's distinct values. Caught by requiring the sharded path to reproduce committed serial counts *exactly* before being pointed at new depth.
2. **Shard-local record indices** made hit records disagree with the merged value array. Caught by an existing REBUILD MISMATCH assertion. Without it the null would have been computed against misaligned records.
3. **A holdout leak**: two hard-coded target lists re-added `koide_Q_1ep` by name after the dataset-level exclusion. (Fixed — and §6 now shows the name guard was never sufficient.)

A fourth, in the opposite direction: applying the holdout filter unconditionally *broke the replay gate*, which must sweep the full historical list to reproduce committed ground truth. Suppressing a guard to satisfy a methodological rule is its own failure mode.

10. What this does and does not establish

Does. No expression built from these two germs at depth ≤ 10 matches any of 19 SM targets with surplus information over chance, exhaustively. Depths 6–10 are proven empty under one engine. Depths 3–5 are empty under the earlier forced engines (depth 4 structurally). Single-target matching in searches of this form is uninformative past a computable depth (~11–13 here), so most such searches cannot conclude anything even in principle. And the natural remedy — simultaneous multi-target matching scored in bits — **must be calibrated against a permutation null**, because the analytic rule prices an impossible object and because chance alone reaches ~11 of 18 independent targets in this vocabulary.

Does not. Say anything about depth 11+ or other germ vocabularies. Constitute new statistics. Provide a validated holdout (§6). Bear on the framework whose germs were used: its acceleration scale, Z , the response sign and the gate frequency remain postulated, not derived, and this search neither supports nor damages them.

Honest scale. One increment on a wall that already had six supports, at substantial engineering cost, in a search whose prior was low and remains low. Its value is that it *closes* rather than leaves ajar — and, unexpectedly, that the stopping rule we proposed for searches of this kind turned out to be wrong in a way worth publishing.

11. v3 audit: what changed and how it was checked

Every sentence of v2 containing a number (108 claims) was checked by independent reviewers against the files the scripts wrote; 26 of them carried only context digits (dates, labels). The coordinating session re-checked each flag against those files. CLAIM_MATCH.py enforces the result: every claim needs a passing review tied to the hash of the source file it cites, so editing a claim or regenerating an output forces a re-review. The details are in ERRATA_v2_CLAIM_MATCH_2026-10-08.md. Six audit scripts whose outputs had never been saved were re-run and their outputs kept, and the depth-4 sweep was re-run. Against the copy committed with v2 (zimmerman-formula cfb6007f), the regenerated expected_count_audit.json differs only in the $\sin^2 \theta_W$ row's recorded

count, which is now compared ($72 = 72$). `holdout_airtight.py` reports 23/26 and labels its last failure FATAL. Two of the failures restate the known `r_tau_mu` omission (§6). The third finds the held-back names inside two scripts, which the audit classes as campaign code: `calibration/charged_lepton_search.py`, a June calibration, and `kappa_rerun/regression_published.py`. Nothing in the repo stops either one from being run as a search on a held-back target. Neither is invoked by `grind.py`, `run_atomos.py`, `VERIFY_ALL.sh` or the campaign shell scripts, and the holdout is withdrawn anyway (§6). The same audit shows that the reported depth-8 and depth-9 runs, which predate the holdout fix, did sweep both holdout keys: `koide_Q_lep` 4 and 6 hits, `r_tau_mu` 1 and 8, none certified. Their clean nulls therefore include the holdouts, while the depth-10 headline excludes them. None of the corrections touches the null, the gate, or any committed verdict.

References

Gross, E. & Vitells, O. 2010, *Eur. Phys. J. C* **70**, 525 — trial factors / look-elsewhere. Koide, Y. 1981 — the charged-lepton mass relation $Q = 2/3$. Milgrom, M. 1999, *Phys. Lett. A* **253**, 273 — the kernel and identity the framework’s germ derives from. Wilczek, F. — *Scaling Mount Planck*, Physics Today (dimensional-analysis numerology, context). CODATA 2022 — $m_p/m_e = 1836.152673426(32)$, $1/\alpha$, a_e .